THE DIGITAL SKETCHBOOK

Hand-Drawn Note-Taking App

Product concept, screen structure, distinctive features, Flutter architecture, and implementation roadmap

Product vision
Capture anything. Connect your thoughts. Find it whenever you need it.

Working name: Scribble

A warm, hand-drawn digital notebook with cream paper surfaces, pencil-like outlines, imperfect ink strokes, small doodles, OCR scanning, searchable notes, and a visual canvas for connecting ideas.

Document purpose

This document translates the product idea into a practical first-release plan. It prioritizes a dependable note-capture workflow before adding advanced AI, collaboration, or cloud features.

DESIGN PRINCIPLE

Make it feel like opening a physical notebook—without losing the speed and searchability of a digital tool.

1. Product vision and goals

Concept. A general-purpose note-taking app for everyday thoughts, scanned documents, project notes, and visual ideas. Its identity comes from a hand-drawn notebook aesthetic, while its value comes from capturing and retrieving information quickly.

Primary goals

- Capture a thought quickly without forcing the user to choose a folder first.
- Scan printed or handwritten content into editable, searchable text.
- Organize notes with collections, tags, favorites, and links.
- Connect related ideas on a flexible visual canvas.
- Keep core note-taking available offline.
- Make the interface expressive but readable and accessible.

Product principle: prioritize capture and retrieval. AI and cloud services should enhance the notebook, not prevent basic use.

2. Visual identity and UI rules

Use warm cream paper as the base, charcoal ink for text, sage green for primary accents, and pencil yellow for highlights. Apply sketch effects selectively: outlines, separators, stickers, and doodles should feel handmade without making controls hard to understand.

Role	Color	Hex
Paper	Warm cream	#F6F0E3
Ink	Soft charcoal	#292B28
Accent	Sage green	#789477
Highlight	Pencil yellow	#E8C875

UI rules

- Use slightly imperfect outlines, not random distortion on every component.
- Use a readable body font; reserve handwritten fonts for headings and small labels.
- Keep paper grain and ruled lines subtle.
- Do not rely on doodles or color alone to communicate an action or status.
- Keep touch targets comfortable and contrast strong enough for accessibility.
- Allow users to reduce decorative effects if they prefer a cleaner interface.

3. Screen structure

Screen	Purpose and key elements
Home — Your desk	Recent and pinned notes; search bar; quick actions for writing, scanning, drawing, and recording; sketch-style note cards; bottom navigation.

Screen	Purpose and key elements
Note editor — Your notebook	Formatted text, headings, lists, checkboxes, drawings, highlights, attachments, tags, favorites, autosave, undo/redo.
Scanner — Capture anything	Camera capture, page-edge detection/cropping, multi-page reorder, OCR preview, editable extracted text, original-image retention.
Canvas — Think visually	Movable note cards, drawing tools, shapes, arrows, labels, zoom/pan, persistent positions and connections.
Search — Find anything	Search titles, note bodies, and OCR text; filter by tags, collections, and dates; jump to matching notes.
Collections	Notebook-like covers for projects, work, personal notes, study, and ideas; tags and favorites.
Settings and backup	Theme/decorations, privacy, export, backup/restore, storage management, and future sync settings.

Navigation recommendation: Home, Search, Canvas, and Collections in the bottom navigation. A prominent + action opens a small menu for New Note, Scan Document, Draw, or Record. Keep settings and trash in a secondary menu.

4. Distinctive features

Feature	Description	Priority
Smart Capture / OCR	Turn photos of documents, book pages, slides, and suitable handwriting into editable text. Keep source images and extracted text together.	MVP
Full-text search	Search typed notes and OCR-extracted content, not just note titles.	MVP
Idea Inbox	Capture quickly; suggest tags or collections later rather than interrupting the user.	MVP
Visual canvas / Idea Threads	Arrange notes, draw connectors, and explain relationships between ideas.	Core differentiator; start simple
Notebook Companion	Optional AI summaries, simpler explanations, title suggestions, and action-item extraction. Require confirmation before modifying notes.	Later
Personal Sketchbook	Paper styles, ink colors, doodle stickers, and collection covers.	MVP basics; expand later
Tasks and reminders	Convert selected text into tasks with dates and links back to the source note.	Later
Version history	Restore earlier versions or recover edited content.	Later
Export and backup	Export to PDF/Markdown/images and provide reliable backup/restore.	Before or soon after launch
Cloud sync	Optional multi-device sync after local data and conflict handling are stable.	Later

OCR note: printed-text recognition and handwriting recognition are not identical. Results depend on image quality, writing style, language, and layout. Always let users review and correct extracted text.

5. Core user flows

Flow A — Capture and find a scanned note

- Tap +, then choose Scan Document.
- Capture one or more pages; crop and reorder them.
- Run on-device OCR and display the extracted text for review.
- Correct mistakes and save as a note with the original scan attached.
- Search for a word that appears on the page and open the result.

Flow B — Capture a quick idea

- Tap + and choose New Note.
- Type a title or start writing immediately.
- Autosave locally; add a tag or collection if desired.
- Return to Home without needing a manual save step.

Flow C — Connect related ideas

- Open Canvas and add existing notes as cards.
- Move cards, draw arrows, and label relationships.
- Save positions and connections locally.
- Tap a card to open the full note.

6. Flutter technology stack

Layer	Recommendation	Reason
Framework	Flutter + Dart	Single codebase and custom UI control.
State management	Riverpod	Explicit, testable state and dependency management.
Local data	Drift + SQLite	Structured notes, tags, collections, links, and transactional updates.
Search	SQLite FTS5 where supported by the chosen setup	Full-text search across note content and OCR text.
OCR	Google ML Kit Text Recognition	On-device recognition for supported scripts and platforms.
Camera / capture	Camera plugin plus a document-capture/cropping workflow	Capture pages and retain image files.
Image storage	App-private file storage	Avoid putting large image blobs directly in ordinary note records.
Drawing	CustomPainter for basic drawing; evaluate a maintained canvas package for advanced interactions	Start small and grow only when needed.

Layer	Recommendation	Reason
Preferences	SharedPreferences	Simple settings such as theme and decoration preferences.
Cloud sync (later)	Supabase or another backend	Authentication and multi-device synchronization after MVP.
AI (later)	A secure backend calling the selected AI provider	Avoid embedding private provider secrets in the mobile app.

Architecture outline

Flutter UI	Riverpod	Repositories
Home, Editor, Scanner, Canvas, Search	Providers, state, commands	Notes, collections, canvas data
Local database	**File storage**	**Services**
Notes, tags, links, OCR text	Original scans, attachments	OCR, indexing, export; sync/AI later

7. Suggested project structure

```
lib/
■■■ app/
■ ■■■ app.dart
■ ■■■ router.dart
■ ■■■ theme/
■ ■■■ app_colors.dart
■ ■■■ app_theme.dart
■ ■■■ sketch_styles.dart
■■■ core/
■ ■■■ database/
■ ■■■ storage/
■ ■■■ search/
■ ■■■ widgets/
■■■ features/
■ ■■■ home/
■ ■■■ notes/
■ ■ ■■■ data/
■ ■ ■■■ domain/
■ ■ ■■■ presentation/
■ ■■■ scanner/
■ ■■■ canvas/
■ ■■■ collections/
■ ■■■ search/
■ ■■■ settings/
■■■ main.dart
```

Organize by feature. Keep UI, domain logic, and data access separate where complexity warrants it. Put shared database, file-storage, and search services in core. Avoid creating abstractions that do not solve a current problem.

8. Initial data model

Entity	Suggested fields
Note	id, title, content, createdAt, updatedAt, isPinned, isArchived, collectionId (optional), sourceType, deletedAt (optional)
Attachment	id, noteId, localPath, mimeType, createdAt, pageIndex (optional), OCR status
Tag	id, name, color (optional)
NoteTag	noteId, tagId
Collection	id, name, description (optional), coverStyle, createdAt
Canvas	id, name, createdAt, updatedAt
CanvasItem	id, canvasId, noteId (optional), itemType, x, y, width, height, content/style data
CanvasConnection	id, canvasId, fromItemId, toItemId, label (optional)

Use stable IDs and database transactions for related updates. Store image paths and metadata in the database while keeping the actual image files in app-private storage. Plan deletion and backup behavior early so notes do not become detached from their attachments.

9. Implementation roadmap

Phase	Work	Deliverable
Weeks 1–2: Foundation	Flutter setup, theme, navigation, Home/Editor/Search/Collections shells, database, note CRUD, autosave.	A working offline notebook for typed notes.
Weeks 3–4: OCR	Camera capture, crop/reorder pages, ML Kit integration, OCR review/edit screen, image storage.	Scanned pages become editable notes.
Weeks 5–6: Search + Canvas	Index titles/body/OCR text; filters; basic canvas, movable cards, drawing and connections.	Searchable notes and first visual-thinking workflow.
Weeks 7–8: Polish	Backup/export, permission and error states, accessibility, real-device tests, app icon and store assets.	Release candidate for Android testing.
After launch	AI assistance, reminders, version history, sync, advanced canvas and collaboration based on feedback.	Incremental improvements guided by actual use.

Scope guard: eight weeks is a planning estimate, not a guarantee. Advanced freeform canvas interactions and reliable cross-device sync can each become substantial projects. Keep the first canvas deliberately simple.

10. Quality, privacy, and release checklist

Core acceptance criteria

- A note can be created, edited, closed, and reopened without losing saved changes.
- The app remains usable offline for core note operations.
- Scanned image files and extracted text remain linked after app restart.
- OCR output can be corrected before or after saving.

- Search finds matches in both typed content and extracted scan text.
- Deleting or archiving notes has predictable behavior, with recovery where supported.
- Canvas positions and connections persist after restart.
- Permission denial, failed capture, unreadable image, and low-storage cases show helpful feedback.

Privacy and safety

- Explain camera and file access when the user invokes those features.
- Keep private notes and scans in app-private storage by default.
- If AI is added, explain what content is sent to a service and obtain appropriate consent.
- Do not ship provider API secrets inside the Flutter app.
- Provide a clear export/backup path and explain what happens when the app is uninstalled.
- Publish an accurate privacy policy before public distribution.

Release checklist

- Test on multiple real Android devices and screen sizes.
- Check performance with large scans and long notes.
- Verify database migrations and backup/restore.
- Prepare app icon, screenshots, store listing, and support contact.
- Collect early feedback and prioritize bugs before adding more features.

11. First development milestone

Build three screens first: Home, Note Editor, and OCR Scanner. Start with sample data, then connect the local database and scanner. Add the canvas only after the capture-and-search workflow works reliably.

Step	End-to-end test
1	Open app and tap +
2	Choose Scan Document
3	Capture and crop a page
4	Review and correct OCR text
5	Save the note with its original scan
6	Search for a word on the scanned page

Success looks like this:
A user can capture a page, correct the text, save it, and find it later in seconds—even without an internet connection.

Working name: Scribble • Platform: Flutter • Initial target: Android • Product direction: offline-first, OCR-enabled, visually expressive notes